SOFTWARE REQUIREMENTS SPECIFICATION

FOR

ONLINE FOOD DELIVERY SYSTEM

Prepared By: Shweta Rahate

Section: B4 Batch: B2 Roll No: 23

Programme: B.Tech Computer Science and Engineering (Data Science & Analytics)

Ramdeobaba University, Nagpur

Submitted To: Prof. Nikita Jamgade

Department: Data Science

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Introduction
2. Overall Description
3. Functional Requirements
4. Non-Functional Requirements
5. External Interface Requirements
6. System Models / UML Diagrams
7. Project Planning
8. Cost Estimation using COCOMO
9. Risk Management
10. Testing
11. Constraints and Assumptions
12. Future Scope
13. Conclusion

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the proposed Online Food Delivery System. It explains the main features, user roles, system requirements, interfaces, UML models, project schedule, cost estimation, risks and testing approach.

1.2 Problem Statement

Traditional food ordering through phone calls or separate restaurant counters can be slow and may lead to incorrect orders. Customers may not know item availability or delivery status, while restaurants and delivery partners need a common way to coordinate orders. The proposed system brings these activities together on one platform.

1.3 Objectives

- Allow customers to browse restaurants, select food and place orders online.
- Support cart management, payment, order tracking and order history.
- Allow restaurants to manage menus and process incoming orders.

- Allow delivery partners to update pickup and delivery status.
- Provide administrators with user, restaurant and reporting controls.

2. Overall Description

2.1 Product Perspective

The system is planned as a web and mobile application with a client interface, application logic, a database and integrations with external services such as payment, maps and notifications.

2.2 User Classes

User	Main activities
Customer	Register, browse menus, manage cart, place and pay for orders, track delivery, view history and rate orders.
Restaurant Owner	Manage restaurant details and menu items; accept/reject orders and update preparation
Delivery Partner	View assigned deliveries and update pickup and delivery progress.
Administrator	Manage users, restaurants, delivery partners, discounts and reports.

2.3 Main System Functions

- Account registration and login
- Restaurant and food search
- Cart and checkout
- Order placement and payment
- Order status and delivery tracking
- Restaurant and delivery management
- Ratings, reviews and administration

3. Functional Requirements

ID	Requirement
FR-01	The system shall allow users to register, log in and log out.
FR-02	The system shall allow customers to search restaurants and browse menus.
FR-03	The system shall allow customers to add, update and remove items from the cart.
FR-04	The system shall validate cart items, delivery address and availability before checkout.
FR-05	The system shall create an order ID and show order confirmation after successful order placement.
FR-06	The system shall support payment status updates and allow a retry when payment fails.
FR-07	The system shall display order progress from confirmation to delivery.
FR-08	The system shall allow restaurant owners to manage menu items and process orders.
FR-09	The system shall allow delivery partners to view assigned deliveries and update delivery status.
FR-10	The system shall maintain order history and allow customers to rate delivered orders.
FR-11	The system shall allow administrators to manage users, restaurants, delivery partners and discounts.

4. Non-Functional Requirements

Quality Attribute	Requirement
Performance	Login and main pages should load quickly; order updates should appear with minimal delay.
Security	Passwords shall be stored as secure hashes. Role-based access shall restrict users to permitted functions.
Reliability	The system should handle invalid input and service errors without losing order records.
Usability	The interface should be simple, consistent and easy to use on common devices.
Availability	The system should be available during normal operating hours, with planned maintenance communicated.
Maintainability	The application should use modular components so updates and bug fixes are manageable.
Data Integrity	Order, payment and delivery records should remain consistent.

5. External Interface Requirements

5.1 User Interface

The interface will provide separate screens or dashboards for customers, restaurant owners, delivery partners and administrators. Forms, menus, cart views, order status screens and management pages will be provided according to each role.

5.2 Hardware Interface

Users will access the system through a computer, smartphone or tablet with a supported browser or mobile application. Server and network resources will be required to host the application.

5.3 Software Interface

The proposed system may use a relational database such as MySQL or PostgreSQL and external APIs for payment processing, maps/geolocation and notifications.

5.4 Communication Interface

Client-server communication is expected to use HTTPS. REST APIs with JSON may be used for application requests, while secure WebSocket communication may be used for live status updates.

6. System Models / UML Diagrams

The following UML diagrams describe the main actors, workflows, system structure, interactions, order lifecycle, deployment and software components.

6.1 Use Case Diagram

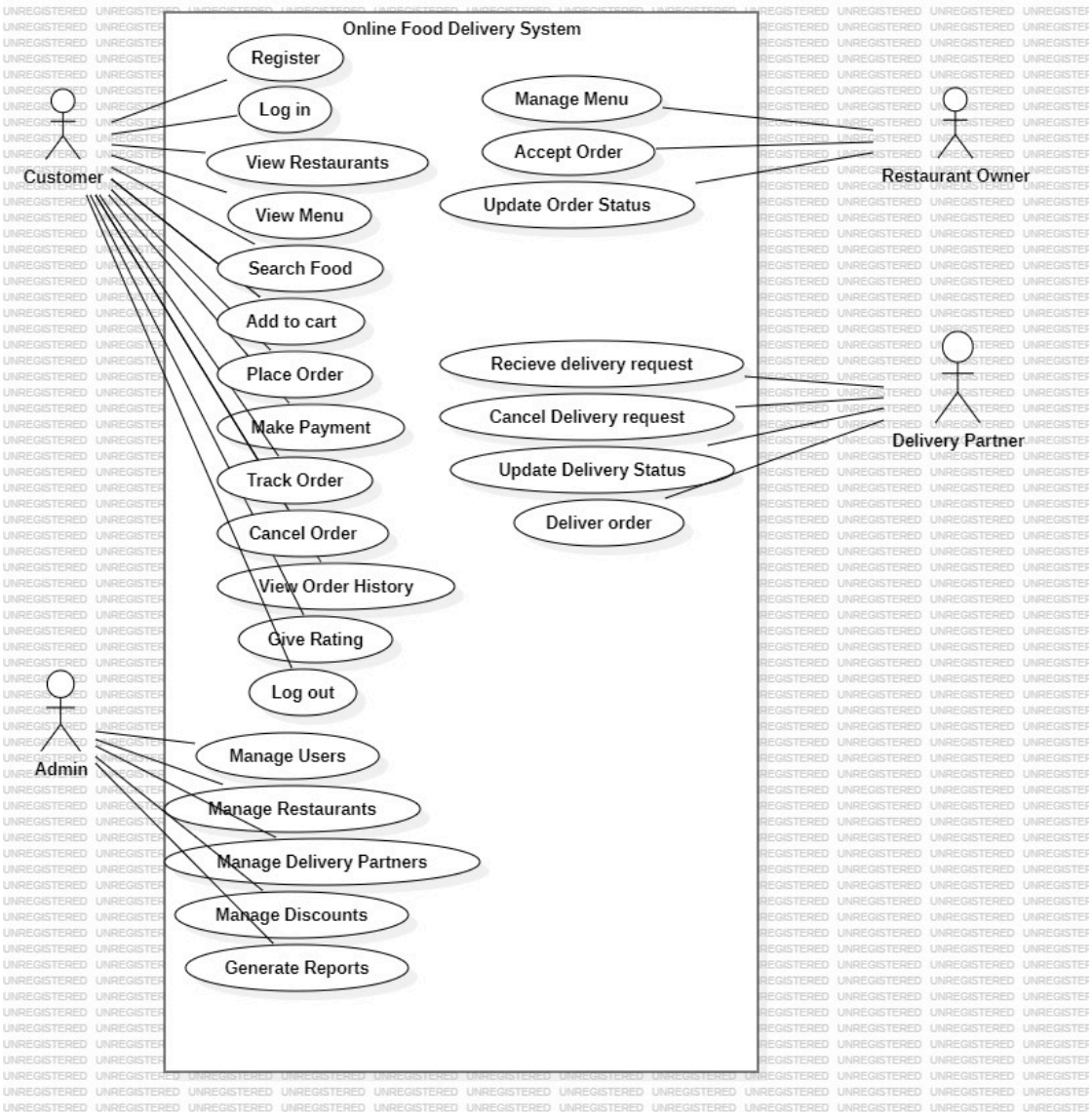


Figure 1. Use Case Diagram

6.2 Activity Diagrams

6.2.1 Activity Diagram – Login

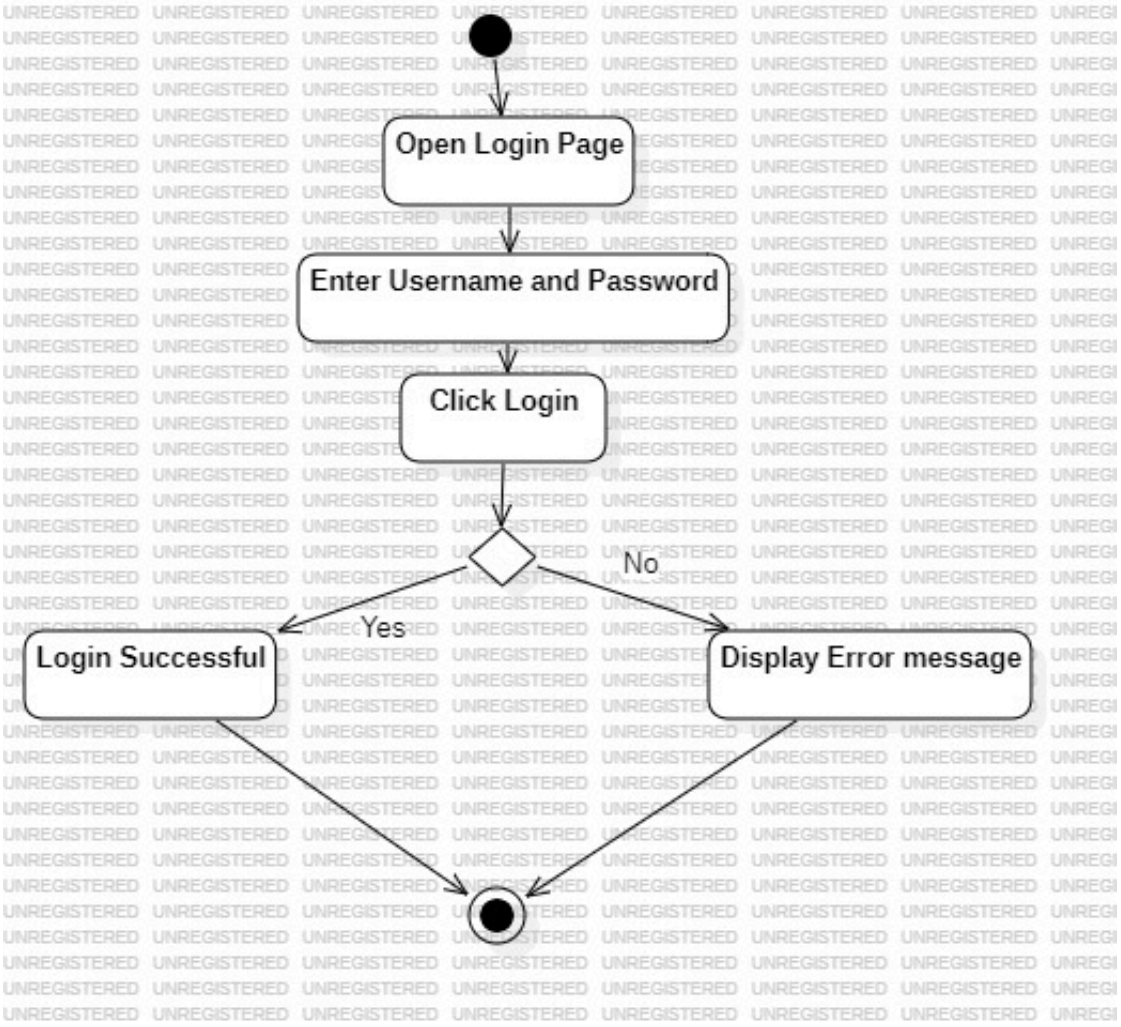


Figure 2. Activity Diagram – Login

6.2.2 Activity Diagram – Place Order

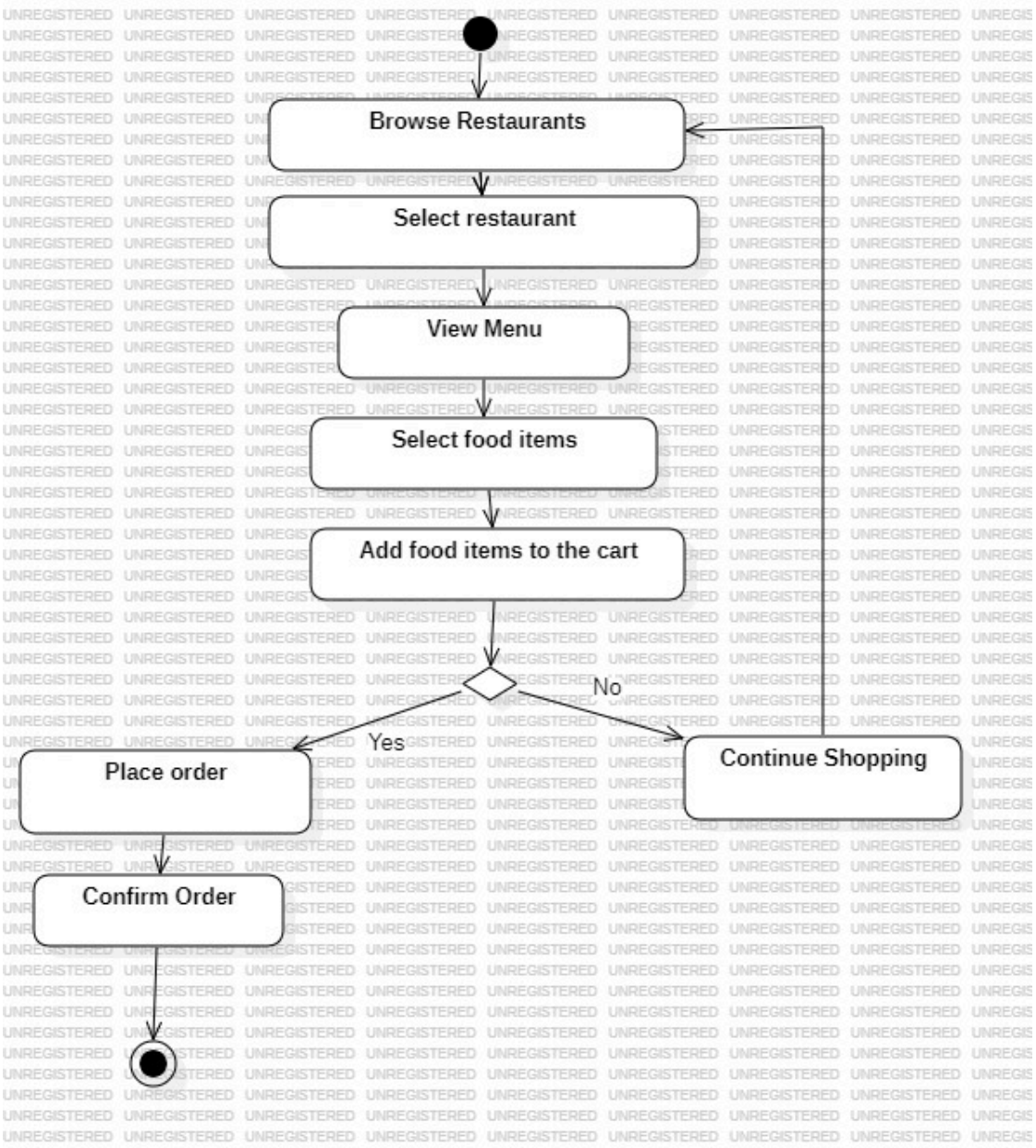


Figure 3. Activity Diagram – Place Order

6.2.3 Activity Diagram – Payment

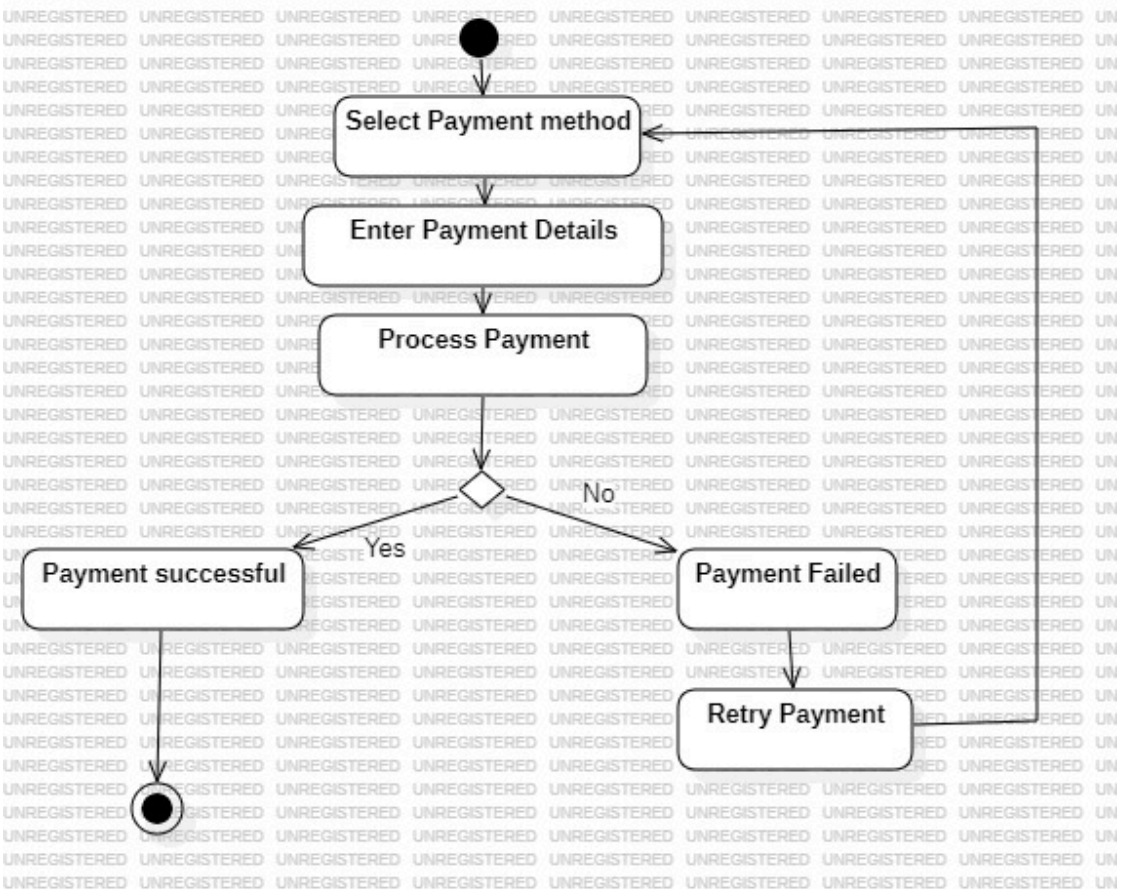


Figure 4. Activity Diagram – Payment

6.3 Sequence Diagrams

6.3.1 Sequence Diagram – User Login

1] User Login Sequence Diagram

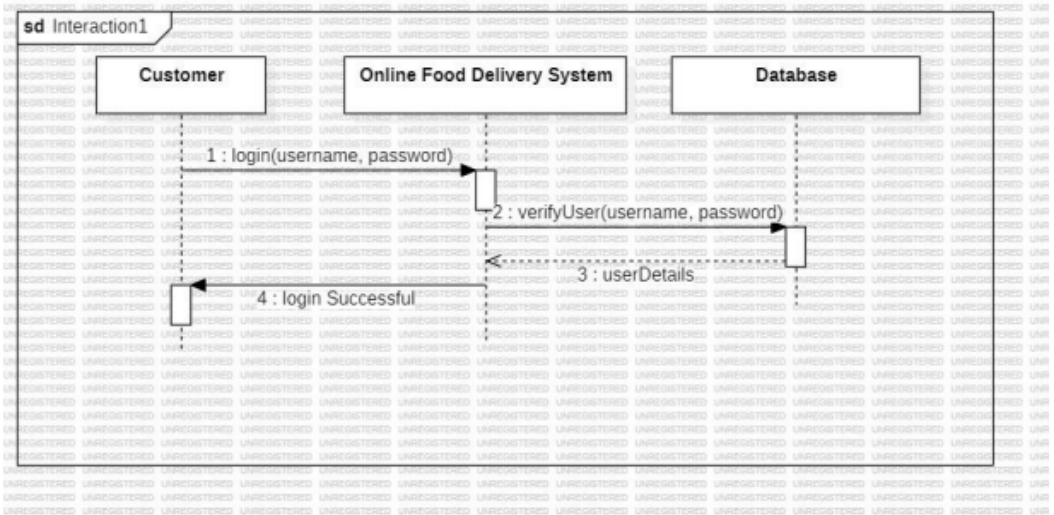


Figure 5. Sequence Diagram – User Login

6.3.2 Sequence Diagram – Place Order

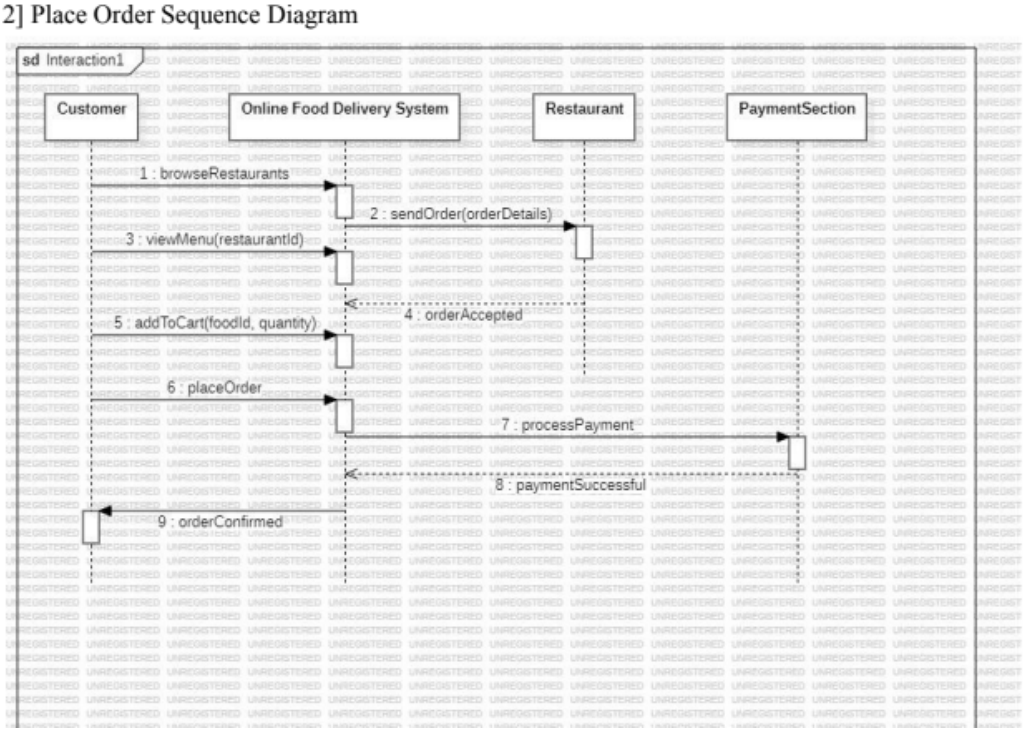


Figure 6. Sequence Diagram – Place Order

6.4 Class Diagram

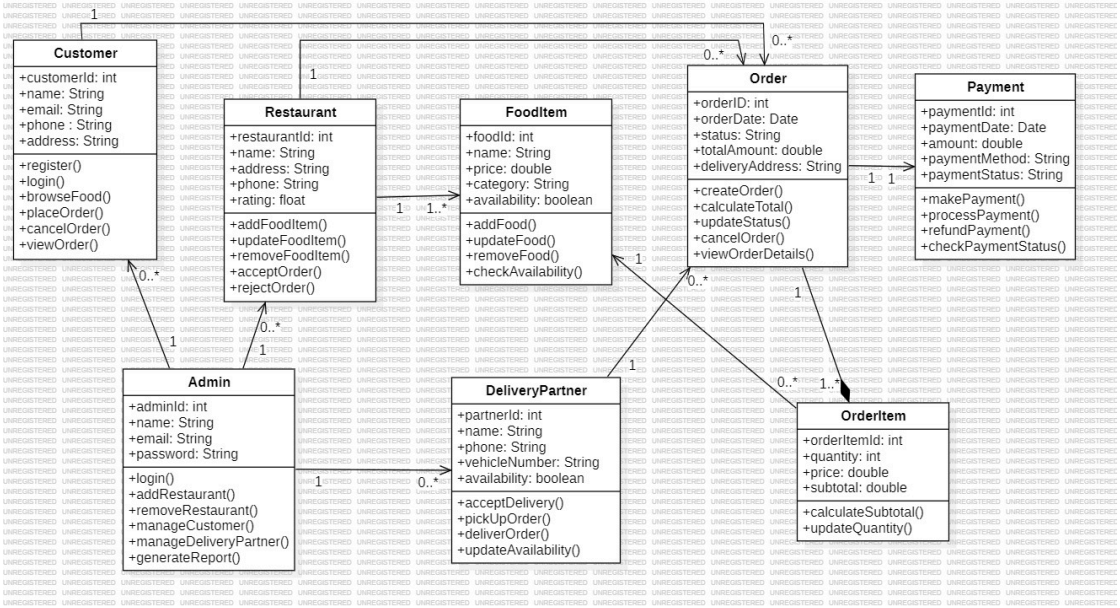


Figure 7. Class Diagram

6.5 State Machine Diagram – Order Lifecycle

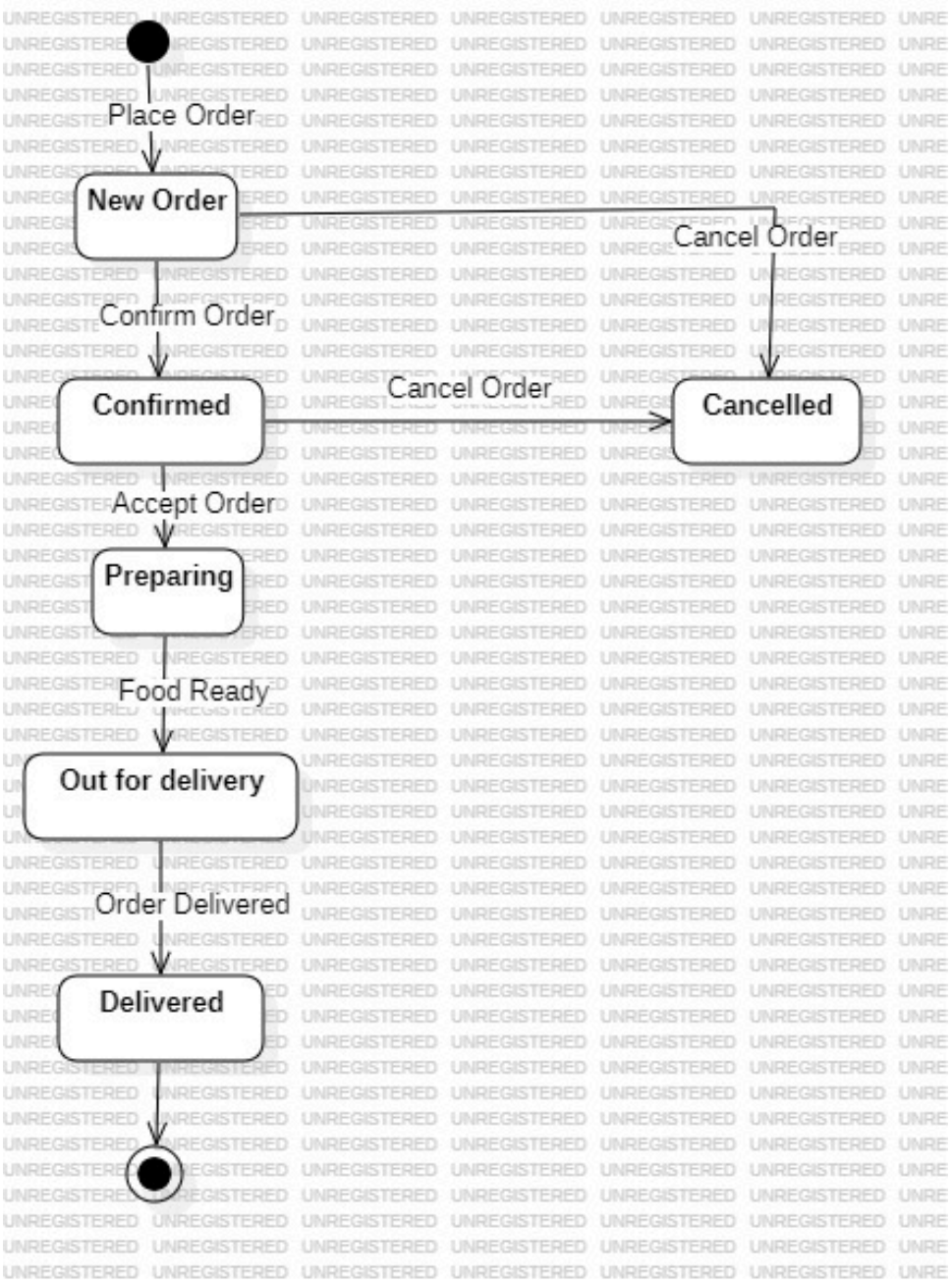


Figure 8. State Machine Diagram – Order Lifecycle

6.6 Deployment Diagram

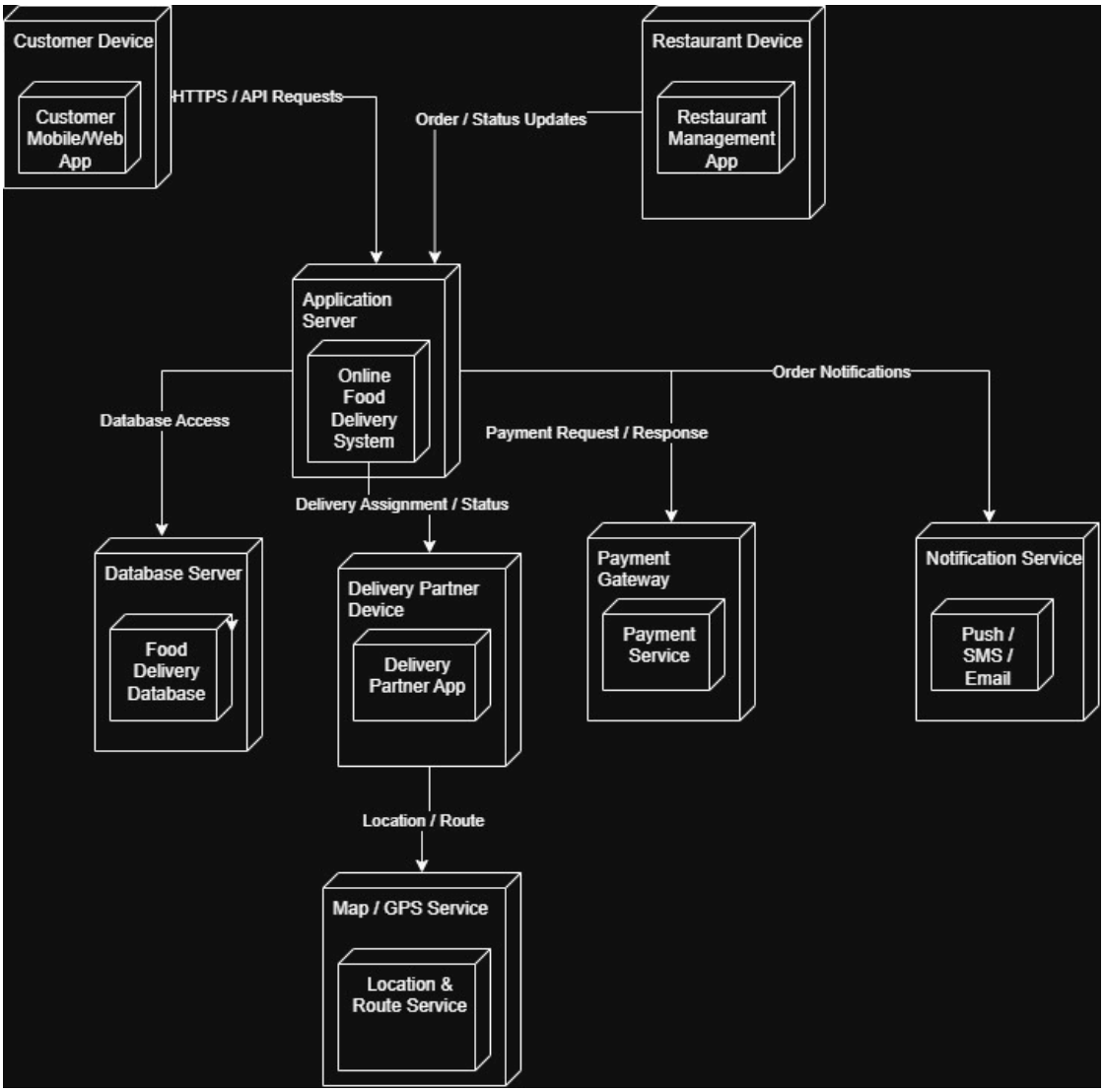


Figure 9. Deployment Diagram

6.7 Component Diagram

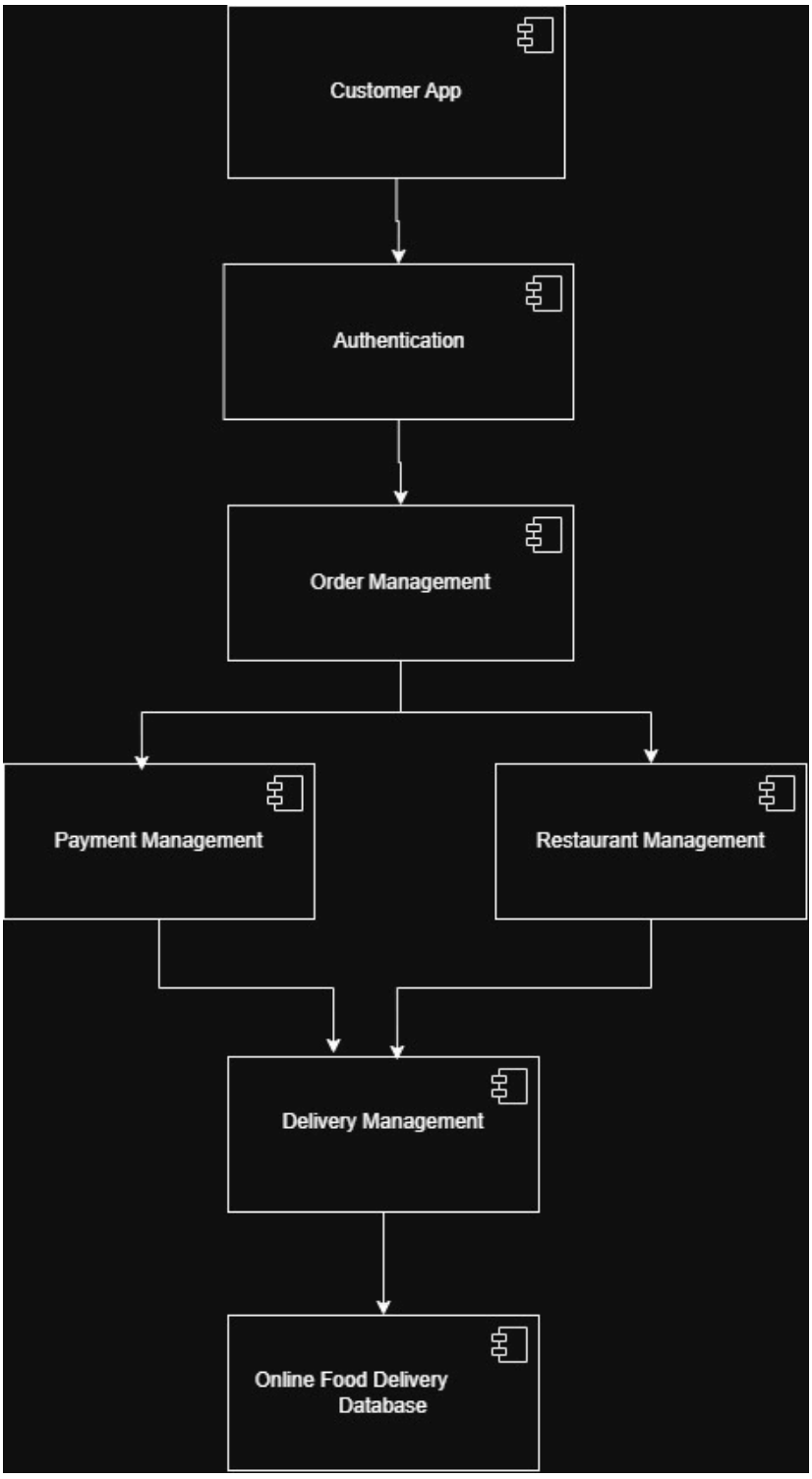


Figure 10. Component Diagram

7. Project Planning

The project is planned in phases from scope definition and requirements gathering through design, implementation, testing and documentation. The schedule below is adapted from the original project plan.

Task ID	Tasks	Start Date	End Date	Duration	Progress
T-01	Project Planning & Scope Definition	15-07-2026	17-07-2026	3	100%
T-02	Feasibility Study & Requirement Gathering	18-07-2026	22-07-2026	5	100%
T-03	Requirement Analysis & SRS	23-07-2026	29-07-2026	7	100%
T-04	System Design & Architecture	30-07-2026	05-08-2026	7	100%
T-05	UI/UX Design	02-08-2026	08-08-2026	7	100%
T-06	Database Design	05-08-2026	09-08-2026	5	100%
T-07	Use Case, Activity & Sequence Modeling	05-08-2026	12-08-2026	8	100%
T-08	Class, State, Component & Deployment Modeling	10-08-2026	16-08-2026	7	100%
T-09	User Module Development	13-08-2026	22-08-2026	10	80%
T-10	Restaurant & Menu Module	20-08-2026	27-08-2026	8	70%
T-11	Cart & Order Management Development	25-08-2026	02-09-2026	9	60%
T-12	Payment Module Development	31-08-2026	06-09-2026	7	40%
T-13	Order Tracking & Review Module	04-09-2026	10-09-2026	7	30%
T-14	Admin Module Development	07-09-2026	14-09-2026	8	20%
T-15	Module Integration	12-09-2026	18-09-2026	7	10%

T-16	Testing & Debugging	17-09-2026	24-09-2026	8	0%
T-17	System Validation & Performance/ Security Checks	23-09-2026	27-09-2026	5	0%
T-18	Final Documentation & Report Preparation	25-09-2026	01-10-2026	7	0%
T-19	Final Review, Presentation & Submission	02-10-2026	04-10-2026	3	0%

Gantt Chart

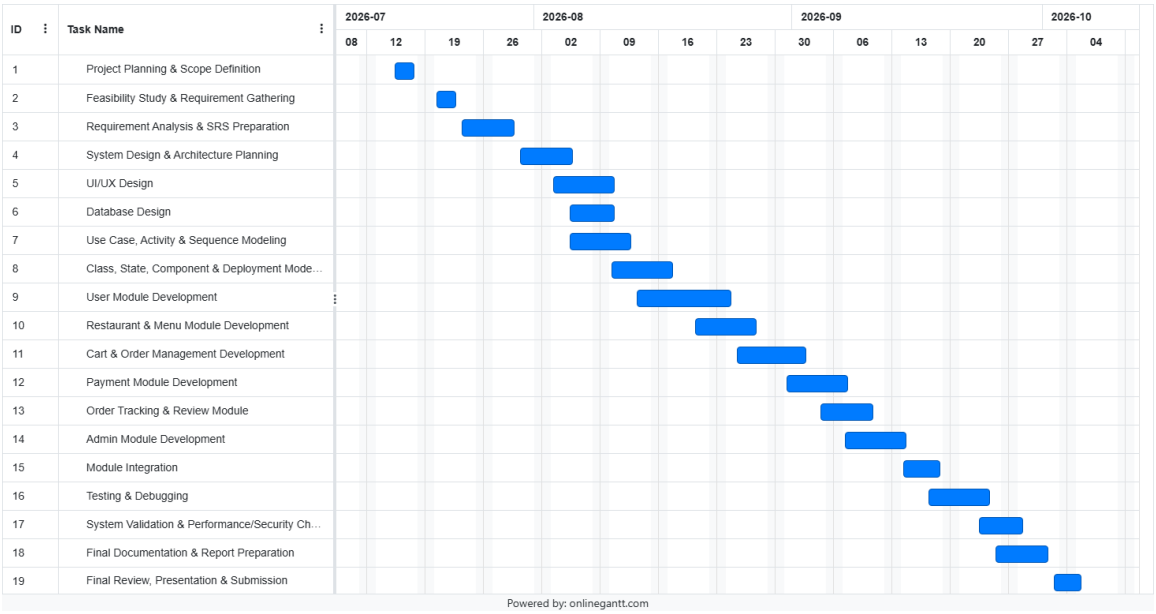


Figure 11. Project Development Schedule – Gantt Chart

8. Cost Estimation using COCOMO

The Basic COCOMO model estimates software development effort from project size in KLOC (thousand lines of code). The provided Python program asks for the project mode, project size and cost per person-month, then calculates effort, development time, staff and estimated cost.

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Formulas used:

- Effort (person-months) = a × (KLOC)^b
- Development time (months) = c × (Effort)^d
- Average staff required = Effort ÷ Development time
- Estimated cost = Effort × Cost per person-month

The final estimate depends on the actual KLOC and the cost per person-month entered into the COCOMO program. These values should be entered based on the project assumptions rather than treated as fixed values.

Organic Mode — COCOMO Cost Estimation

Parameter	Value
Project Size	10.0 KLOC
Effort	26.93 Person-Months
Development Time	13.55 Months
Staff Required	3.08 Persons
Estimated Cost	Rs. 1346422.15

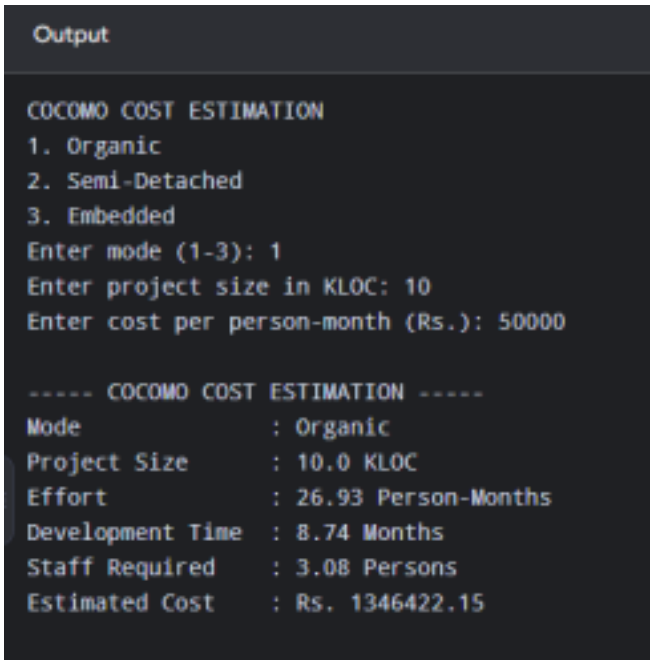


Figure 12. COCOMO Organic Mode Output

Semi Detached Mode — COCOMO Cost Estimation

Parameter	Value
Project Size	10.0 KLOC
Effort	39.55 Person-Months
Development Time	9.06 Months
Staff Required	4.37 Persons
Estimated Cost	Rs. 1977385.11

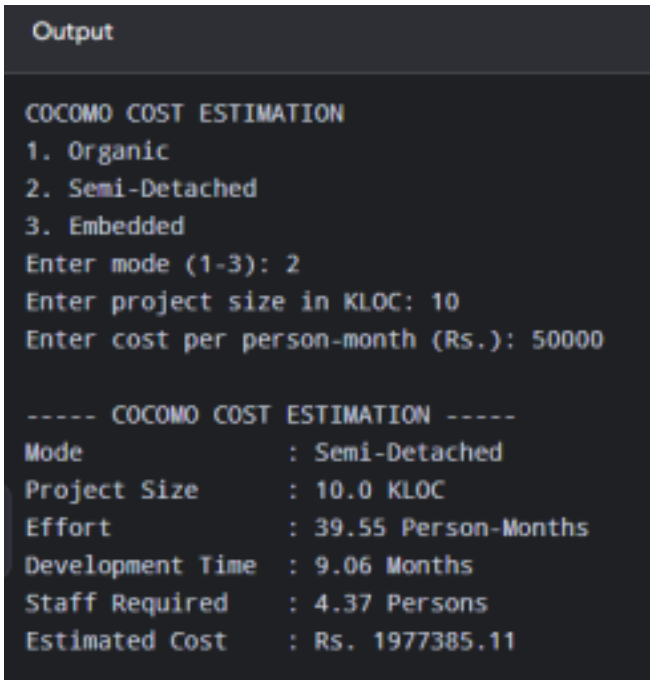


Figure 13. COCOMO Semi-Detached Mode Output

Embedded Mode — COCOMO Cost Estimation

Parameter	Value
Project Size	10.0 KLOC
Effort	57.06 Person-Months
Development Time	9.12 Months
Staff Required	6.26 Persons
Estimated Cost	Rs. 2852807.75

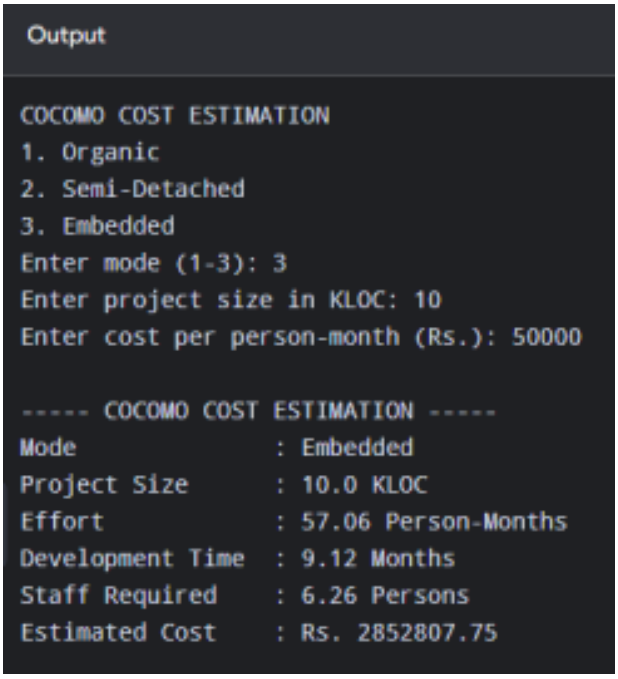


Figure 12. COCOMO Embedded Mode Output

9. Risk Management

Risk	Impact / Likelihood	Mitigation
Requirement changes	Medium	Confirm requirements early and review changes with the project guide.
Payment gateway failure	Medium	Provide clear failure messages and allow retry or alternate payment methods.
Data loss or corruption	High	Use database backups and validate updates to important records.
Unauthorized access	High	Use secure password storage, role-based access and HTTPS.
Map or notification service unavailable	Medium	Show the latest available status and handle service errors gracefully.
Schedule delay	Medium	Track progress against the Gantt chart and prioritize core features.

10. Testing

Testing will check whether the system meets its functional and non-functional requirements. Test cases should be executed with valid, invalid and boundary inputs, and their actual results should be recorded.

Test ID	Test Scenario	Expected Result
TC-01	Register with valid details	Account is created.
TC-02	Register using an existing email	Duplicate registration is rejected.
TC-03	Log in with valid credentials	Correct user dashboard opens.
TC-04	Add or remove items from cart	Cart and total are updated correctly.
TC-05	Place order with unavailable item	Checkout is blocked with a clear message.
TC-06	Payment gateway declines payment	Payment is marked failed and retry is offered.

TC-07	Customer opens an admin function	Access is denied.
TC-08	Delivery partner marks an order delivered	Order status is updated.

11. Constraints and Assumptions

Constraints

- The system depends on internet connectivity and supported devices.
- Payment, map and notification services depend on third-party APIs.
- Only authorized users may access role-specific functions.
- Actual deployment technology may vary based on implementation resources.

Assumptions

- Users provide valid contact and delivery details.
- Restaurants keep menu information and availability updated.
- Delivery partners have access to a supported device and network connection.
- External service providers supply working APIs and credentials when integrated.

12. Future Scope

- Support multiple languages and currencies.
- Provide personalized food recommendations.
- Improve delivery-time prediction and route optimization.
- Add loyalty points, subscriptions and promotional offers.
- Enhance analytics and reporting for restaurants and administrators.

13. Conclusion

The proposed Online Food Delivery System brings customers, restaurants, delivery partners and administrators into one coordinated platform. This SRS summarizes its requirements, interfaces, UML models, project schedule, COCOMO estimation method, risks and testing approach. It can be used as a reference for future implementation and evaluation.